



Plant Guardian

Aplicación móvil para el cuidado inteligente de plantas de exterior e interior

2no informe de progreso

20/05/2026

Fiorella Lucia Queirolo Chavez

Índice

1. Introducción.....	3
2. Planificación.....	3
2.1. Tercer sprint.....	3
3. Estado de la aplicación.....	4
3.1. Objetivos.....	5
3.1.1. Desarrollar un sistema inteligente de asistencia para el cuidado de plantas...	5
3.1.2. Diseñar una interfaz gamificada centrada en la experiencia de usuario.....	5
3.1.3. Implementar un repositorio de datos centralizado para el seguimiento y la personalización de cada planta.....	6
3.2. Requisitos.....	6
3.2.1. Requisitos funcionales.....	7
3.2.2. Requisitos no funcionales.....	9
3.3. Organización.....	9
3.3.1. Estructura del backend.....	10
3.3.2. Estructura del frontend.....	10
4. Desarrollo de las pantallas.....	10
4.1. Registro e inicio de sesión.....	11
4.2. Pantalla principal de inicio.....	11
4.3. Datos y gestión usuario.....	12
4.4. Añadir planta.....	12
4.5. Mis plantas.....	13
5. Repaso general.....	14
5.1. Resultados obtenidos.....	14
5.2. Pantallas implementadas.....	15
5.3. Conclusiones.....	16
6. Inteligencia artificial.....	17
7. Webgrafía.....	18

1. Introducción

Este segundo informe de progreso recoge los avances realizados en las fases finales del proyecto Plant Guardian. En este punto, el proyecto ha evolucionado desde una idea inicial y un prototipo conceptual hasta convertirse en una aplicación funcional en forma de Producto Mínimo Viable (MVP).

El informe se centra principalmente en el desarrollo del tercer *sprint*, durante el cual se finalizaron las interfaces de usuario y su conexión con el backend. También se analiza el grado de cumplimiento de los objetivos planteados al inicio del proyecto, así como el estado actual de los requisitos funcionales y no funcionales.

Por otro lado, se detalla el alcance actual del proyecto, explicando cómo se han implementado las distintas pantallas y cuál ha sido el resultado final obtenido. Además, se incluye una descripción del uso de herramientas de inteligencia artificial generativa, como Gemini y Claude, que han servido de apoyo tanto en tareas técnicas como en la mejora del diseño y la experiencia de usuario.

En resumen, este informe ofrece una visión general del estado actual del proyecto, los resultados obtenidos y los pasos pendientes antes de la entrega final.

2. Planificación

En el anterior informe de progreso se explicó que el seguimiento del trabajo se realizaba conforme a la planificación establecida en la primera fase del proyecto, basada en una metodología iterativa mediante *sprints*.

Este enfoque se ha mantenido a lo largo del desarrollo, ya que ha permitido organizar el trabajo de forma incremental, priorizando funcionalidades y facilitando la adaptación a posibles cambios o ajustes durante la implementación.

Para la gestión y control de tareas se ha continuado utilizando la plataforma Trello. Gracias a su uso, ha sido posible visualizar en todo momento el estado de cada tarea (en *backlog*, del *sprint* actual, en desarrollo y hechas).

2.1. Tercer sprint

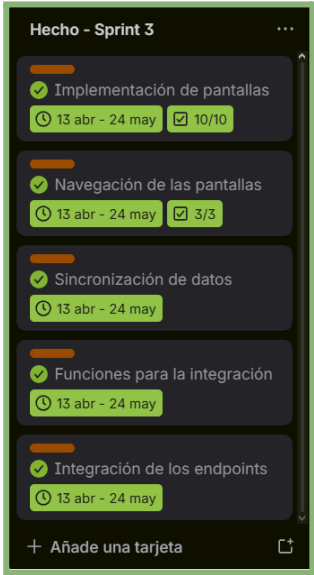
El tercer *sprint* se centró en el desarrollo e implementación de las pantallas de la aplicación a partir del diseño previamente definido en Figma durante la primera fase del proyecto.

En esta etapa, el objetivo principal fue integrar la parte visual de la aplicación con el *backend*, permitiendo que el usuario pudiera interactuar correctamente con todas las funcionalidades desarrolladas.

Durante este *sprint* se implementaron todas las interfaces de la aplicación y se conectaron los distintos *endpoints* del *backend* para garantizar el funcionamiento de cada apartado de la aplicación. Esto incluye tanto el desarrollo del flujo general de

navegación (como las transiciones entre pantallas, animaciones y elementos visuales) como la integración de la lógica funcional necesaria para gestionar las acciones del usuario y la comunicación con el servidor.

Los diferentes flujos y pantallas se detallarán en el apartado 4. No obstante, a continuación se presenta un resumen de las tareas realizadas durante este *sprint*.

Tareas - Sprint 3	
	Actividad 3.1 - Desarrollo del frontend ☒ Tarea 3.1.1 - Implementación de pantallas ☒ Tarea 3.1.2 - Implementación de la navegación ☒ Tarea 3.1.3 - Conexión con las APIs Actividad 3.2 - Integración de la app ☒ Tarea 3.2.1 - Integración de los endpoints ☒ Tarea 3.2.2 - Sincronización de datos

Cabe destacar que, para la implementación de las pantallas, fue necesario organizar el desarrollo en función de las distintas secciones definidas en el diseño inicial. De esta forma, las pantallas se dividieron en los siguientes bloques:

- Registro e inicio de sesión.
- Pantalla principal de inicio.
- Apartado de datos y gestión usuario.
- Proceso de añadir planta (identificación, análisis, rutina personalizada).
- Mis plantas (tareas, lugares, ver planta).

Y para la navegación entre pantallas, se estructuró en tres aspectos: transiciones, animaciones y la conexión entre vistas.

3. Estado de la aplicación

En esta fase del proyecto, la aplicación ha pasado de ser un prototipo a una aplicación funcional. A continuación, se detallan los aspectos que definen su estado actual, incluyendo los objetivos establecidos al inicio del proyecto, los requisitos y funcionalidades previstas, así como la organización final que ha dado forma al desarrollo de la aplicación.

3.1. Objetivos

De los objetivos definidos al inicio del proyecto, se considera que los tres han sido alcanzados, aunque con distintos niveles de desarrollo en cada caso.

A continuación, se presenta la justificación y el grado de cumplimiento de cada uno de ellos, analizando su implementación y los resultados obtenidos durante el desarrollo de la aplicación.

3.1.1. Desarrollar un sistema inteligente de asistencia para el cuidado de plantas

Este objetivo se ha alcanzado mediante la integración de las APIs de Trefle y Gemini, que se utilizan de forma conjunta para implementar un sistema inteligente de asistencia en el cuidado de plantas.

El sistema se ha diseñado como un flujo guiado que permite al usuario registrar y gestionar sus plantas de manera sencilla, mientras el sistema obtiene y procesa la información de forma automática.

El proceso comienza cuando el usuario introduce los datos básicos de una planta dentro de la aplicación. Esta información se utiliza para realizar una consulta en la API de Trefle, que permite identificar la planta de forma precisa. A partir de esta identificación, se recuperan datos estructurados como la especie, sus características principales y sus necesidades de mantenimiento (riego, luz, temperatura, etc.).

Una vez obtenida esta información, se almacena en la base de datos junto con los datos del usuario. Esta asociación permite mantener un seguimiento individualizado de cada planta, relacionándola directamente con su propietario y su contexto de uso.

Posteriormente, esta información es utilizada como entrada para la API de Gemini, que se encarga de generar diagnósticos inteligentes y recomendaciones personalizadas. Estos diagnósticos no se basan únicamente en la especie de la planta, sino que también incorporan variables contextuales proporcionadas por el usuario, como su ciudad y el entorno donde se encuentra la planta. Esto permite ajustar las recomendaciones a condiciones ambientales reales, mejorando su precisión y utilidad.

Finalmente, todo este proceso se ha integrado dentro de un flujo de interacción progresivo, diseñado para minimizar la complejidad percibida por el usuario. De esta forma, el sistema oculta la complejidad técnica de las APIs y la gestión de datos, ofreciendo una experiencia sencilla, guiada e intuitiva durante el registro y seguimiento de plantas.

3.1.2. Diseñar una interfaz gamificada centrada en la experiencia de usuario

Este objetivo no se ha completado en la medida inicialmente prevista. Desde el inicio del proyecto se priorizó que la aplicación se centrara en su funcionalidad principal: la identificación, diagnóstico y cuidado de plantas.

Por este motivo, se decidió limitar la incorporación de elementos de gamificación, ya que un exceso de estos podría restar claridad y desviar la atención del objetivo principal de la aplicación.

No obstante, durante el estudio de mercado se identificó que la gamificación es un elemento relevante y ampliamente utilizado en este tipo de aplicaciones. Por ello, se optó por diseñar una interfaz equilibrada, que evitara sobrecargar al usuario con elementos visuales o mecánicas complejas, pero que al mismo tiempo mantuviera una base preparada para futuras ampliaciones.

En este sentido, la interfaz se ha diseñado siguiendo principios de simplicidad y claridad, priorizando la experiencia de usuario y la facilidad de uso [1] [2]. Aun así, se han incorporado elementos básicos de gamificación [3], siendo el aumento de experiencias según las tareas realizadas y un sistema de logros que el usuario va desbloqueando a medida que utiliza la aplicación, así como indicadores visuales que reflejan su progreso dentro del sistema.

De este modo, se consigue un equilibrio entre una experiencia de uso limpia e intuitiva y una estructura que permite evolucionar hacia un sistema más gamificado en futuras versiones.

3.1.3. Implementar un repositorio de datos centralizado para el seguimiento y la personalización de cada planta

Este objetivo ha sido el más complejo de implementar, ya que ha requerido un análisis detallado sobre la estructura de datos y la organización de la información dentro del sistema.

Tras varios diseños (esto se ve claramente en las modificaciones de los diagramas, así como el diagrama de clases descartado en el segundo *sprint*), se optó por una arquitectura sencilla pero eficiente, adecuada para el MVP del proyecto.

El modelo de datos se basa en entidades principales como usuario, planta, logros y tareas, complementadas por tablas intermedias que permiten establecer relaciones entre ellas. De este modo, es posible asociar las plantas de cada usuario con sus tareas específicas, así como con los logros obtenidos durante el uso de la aplicación.

Por último, además del diseño del modelo de datos, también fue necesario configurar correctamente la base de datos para soportar operaciones CRUD (creación, lectura, actualización y eliminación) [4], garantizando así la correcta gestión de la información.

3.2. Requisitos

A continuación, se describirá el estado de los requisitos funcionales y no funcionales definidos inicialmente durante la fase de planificación del proyecto, junto con las razones por las cuales algunos de ellos no han podido ser completados.

3.2.1. Requisitos funcionales

En relación con los requisitos funcionales, se ha logrado completar la mayoría de ellos. Concretamente, de un total de 18 requisitos definidos, únicamente 3 no han podido implementarse.

A continuación, se presenta un listado con los requisitos desarrollados y, posteriormente, se explican los motivos por los cuales no ha sido posible completar los 3 requisitos restantes.

ID	Requisito	Estado
RF-1-1	El sistema ha de permitir que el usuario pueda iniciar sesión.	☒
RF-1-2	El sistema ha de permitir que el usuario pueda registrarse con su email y su contraseña.	☒
RF-1-3	El sistema ha de permitir la gestión de los datos del usuario.	☒
RF-1-4	El sistema ha de permitir que el usuario pueda iniciar sesión con su cuenta de Google.	☐
RF-2-1	El sistema ha de identificar la planta a partir de una foto.	☒
RF-2-2	El sistema debe realizar un análisis de la planta y mostrar los resultados.	☒
RF-2-3	El sistema ha de guardar la información de la planta.	☒
RF-2-4	El sistema ha de crear una rutina personalizada para cada planta.	☒
RF2-5	El sistema debe mostrar una lista de acciones a realizar en la rutina personalizada.	☒
RF-2-6	El sistema ha de permitir crear colecciones de plantas.	☐
RF-3-1	El sistema ha de obtener la información meteorológica actual según la ubicación del usuario.	☒
RF-3-2	El sistema ha de mostrar la información meteorológica local.	☒
RF-3-3	El sistema ha de ofrecer recomendaciones en las rutinas adaptadas al clima actual.	☒
RF-3-4	El sistema ha de generar alertas ante condiciones climáticas.	☐
RF-4-1	El sistema ha de registrar los puntos (XP) según las acciones realizadas en cada planta.	☒
RF-4-2	El sistema ha de otorgar un nivel de experiencia al usuario.	☒
RF-4-3	El sistema ha de mostrar los logros del usuario en su perfil.	☒

RF-4-4	El sistema ha de otorgar recompensas (iconos de usuario o decoraciones en el perfil).	☒
--------	---	-------------------------------------

Los requisitos RF-1-4, RF-2-6 y RF-3-4 no se han podido completar debido a las siguientes razones:

- **RF-1-4: El sistema ha de permitir que el usuario pueda iniciar sesión con su cuenta de Google.**

Para implementar esta funcionalidad habría sido necesario realizar llamadas a la API de autenticación de Google mediante Firebase. Esto implicaba integrar una API adicional dentro de la aplicación. Aunque no suponía un problema técnico relevante, sí añadía complejidad innecesaria en esta fase del proyecto.

Por este motivo, la gestión avanzada de cuentas de usuario se consideró una funcionalidad secundaria y no prioritaria, ya que puede incorporarse en futuras versiones sin afectar al funcionamiento principal de la aplicación.

- **RF-2-6: El sistema ha de permitir crear colecciones de plantas.**

El sistema de colecciones estaba planteado inicialmente para organizar las plantas según los lugares donde se encontraban ubicadas. Sin embargo, durante el proceso de diseño e implementación de las pantallas, se observó que esta lógica requería añadir nuevas vistas y una pestaña adicional dedicada a la agrupación por ubicaciones.

Tras analizar nuevamente los resultados obtenidos en las entrevistas realizadas a los usuarios, se comprobó que la mayoría disponía de entre 2 y 10 plantas. Debido a esta cantidad tan reducida, la clasificación por lugares no aportaba una mejora significativa en la experiencia de uso, ya que en muchos casos un espacio concreto, como el baño o una habitación, únicamente contendría una planta.

Por este motivo, se decidió sustituir el sistema de colecciones por un buscador. Esta alternativa permitía mantener un diseño más limpio, claro y sencillo de utilizar. Además, facilita que el usuario encuentre rápidamente una planta concreta mediante una búsqueda directa, evitando realizar clics adicionales o navegar entre diferentes pestañas.

- **RF-3-4: El sistema ha de generar alertas ante condiciones climáticas.**

Esta funcionalidad todavía no se ha implementado, ya que no se consideró prioritaria durante la fase actual de desarrollo y, por tanto, no se incluyó en esta versión.

No obstante, tras adquirir experiencia con el entorno de Android Studio y la arquitectura de la aplicación, se considera que su implementación no presenta una complejidad elevada. Por este motivo, es probable que esta funcionalidad se incorpore en futuras versiones del proyecto.

3.2.2. Requisitos no funcionales

En cuanto a los requisitos no funcionales, del total de 7 definidos inicialmente, actualmente solo se han podido implementar 2 de ellos.

De manera general, esta situación se debe principalmente a la dependencia de servicios externos, en concreto a la disponibilidad del servidor de *backend*, el cual todavía no ha sido configurado ni desplegado en un entorno de producción.

La aplicación, por el momento, no se encuentra alojada en un servidor activo, lo que limita la validación completa de varios requisitos no funcionales relacionados con rendimiento, disponibilidad y comunicación con servicios externos.

Para solventar esta limitación, se ha contemplado el uso de la plataforma **Render** [5], que permite el despliegue en un entorno gratuito. Asimismo, se prevé implementar una lógica de mantenimiento mediante llamadas periódicas al servidor con **cron-job** [6] con el objetivo de evitar su suspensión durante el periodo de desarrollo restante [7].

Por este motivo, los requisitos no funcionales pendientes se abordarán en una fase posterior del proyecto, una vez el servidor esté correctamente habilitado. En ese momento, se llevarán a cabo las pruebas necesarias para validar su cumplimiento.

ID	Requisito	Estado
RNF-1-1	La interfaz debe ser intuitiva, visual y amigable con el usuario.	☒
RNF-1-2	La aplicación debe permitir la identificación de una planta en menos de 3 clics desde la pantalla de inicio.	☒
RNF-1-3	La aplicación debe tener un diseño <i>responsive</i> que se adapte a diferentes tamaños de pantalla.	☐
RNF-2-1	El tiempo de respuesta para el reconocimiento de imágenes mediante IA no debe superar los 10 segundos.	☐
RNF-2-2	La aplicación debe ser ligera, con un tamaño de instalación inicial optimizado para no ocupar mucho espacio en el móvil.	☐
RNF-3-1	El sistema debe tener una disponibilidad del 99% del tiempo.	☐
RNF-3-2	La aplicación debe permitir la consulta en modo offline de las fichas de plantas que ya han sido añadidas al inventario del usuario.	☐

3.3. Organización

En cuanto a la organización del trabajo, es decir, la forma en la que se ha estructurado y construido la arquitectura del proyecto, tal y como se especifica en el documento de especificación, se ha seguido un modelo por capas claramente diferenciadas entre *backend* y *frontend*.

A continuación, se describe la organización interna del proyecto en cuanto a su estructura de carpetas. Esta organización se ha ido definiendo de manera progresiva a medida que avanzaba el desarrollo, manteniendo siempre como referencia la planificación inicial establecida.

3.3.1. Estructura del backend

Este módulo se encarga de gestionar las peticiones del cliente, la conexión con la base de datos Supabase y la integración con servicios externos como la API de Trefle y Gemini. Su estructura interna se organiza de la siguiente manera:

- **logic/, routers/ y services/:** Módulos encargados de separar la lógica de negocio, la definición de los *endpoints* y la gestión de servicios externos.
- **database.py:** Configuración de la conexión con la base de datos centralizada.
- **main.py:** Punto de entrada de la API, donde se inicializan y ejecutan los servicios del *backend*.
- **schemas.py:** Definición de los modelos de datos utilizados para la comunicación entre el servidor y la aplicación, son las clases que definen el tipo de atributos que deben tener.
- **requirements.txt:** Listado de dependencias necesarias para el despliegue del *backend*, muy importante para configurar el entorno virtual [8].

3.3.2. Estructura del frontend

El *frontend* constituye la “cara” de la aplicación que verá el usuario cuando lo utilice. Su organización es la siguiente:

- **ui/:** Contiene las distintas pantallas de la aplicación, como registro, inicio, añadir planta y mis plantas.
- **data/:** Gestiona la comunicación asíncrona con el *backend* con los *endpoints* [9] mediante Retrofit [10].
- **tools/:** Incluye adaptadores para listas [11] y *ViewModels* [12], responsables de mantener y gestionar el estado de los datos entre las diferentes pantallas [13].
- **res/:** Carpeta destinada a recursos de la aplicación como *layouts*, imágenes y elementos de interfaz.
- **build.gradle.kts y settings.gradle.kts:** Gestionan las dependencias del proyecto, incluyendo librerías como Coil [14], Retrofit y componentes de Material Design.

4. Desarrollo de las pantallas

A continuación, se detalla la implementación final de las interfaces de usuario durante el tercer *sprint*, centrada en la experiencia de uso (UX), tal y como se especificaba en los objetivos, así como la integración de los datos provenientes del backend.

En todas las pantallas se ha seguido el diseño realizado en Figma, aunque en algunos casos ha sido necesario modificar elementos como colores, formas o, en el caso que se explicará más adelante con mayor profundidad, la organización de la pantalla.

Por lo tanto, en los siguientes apartados, se explicará de forma resumida en qué ha consistido cada pantalla y cómo se ha llevado a cabo su implementación.

4.1. Registro e inicio de sesión

Estas pantallas corresponden a la gestión del acceso y la creación de cuentas, siendo la de inicio de sesión la primera pantalla que el usuario encuentra al acceder por primera vez a la aplicación y, también, cuando cierra sesión.

Básicamente, la creación de la cuenta requiere los siguientes datos: nombre, correo electrónico, ciudad (necesaria para ayudar a Gemini en su diagnóstico), contraseña y una verificación para confirmar que la contraseña es correcta.

Una vez creada la cuenta, ésta se almacena en la base de datos y puede utilizarse posteriormente para iniciar sesión. Al acceder, el identificador del usuario se mantiene en la sesión, lo que permite acceder a sus datos en otras funcionalidades de la aplicación cuando sea necesario.

Las pantallas resultantes se muestran en las siguientes imágenes. El único cambio respecto al diseño de Figma ha sido la modificación de los márgenes de la pantalla y el color del icono utilizado en los botones de navegación y de confirmación.

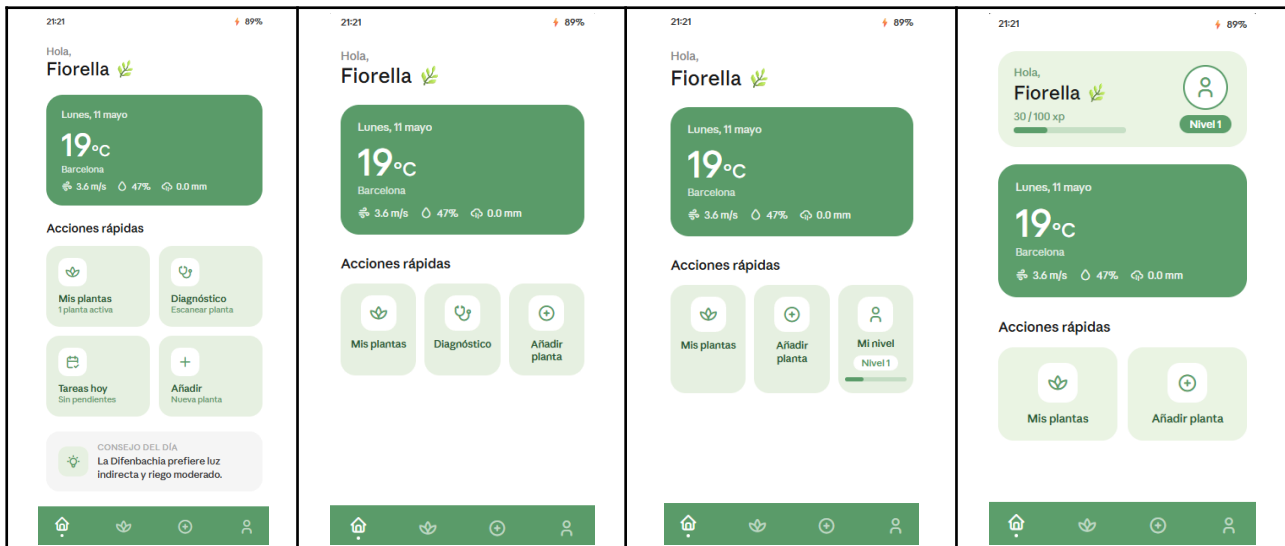
4.2. Pantalla principal de inicio

Para el diseño de esta pantalla fue necesario realizar varias iteraciones y rediseños respecto a la propuesta inicial desarrollada en Figma, ya que existían dudas sobre si el diseño original funcionaba correctamente dentro de la aplicación. Desde el inicio del desarrollo, se percibía que la pantalla resultaba demasiado vacía y que no terminaba de integrarse visualmente con el resto de la interfaz.

Con el objetivo de mejorar este apartado, se utilizó **Claude** como apoyo durante la fase de diseño, especialmente para generar propuestas visuales preliminares antes de trasladarlas al código. Esta herramienta se empleó debido a la necesidad de explorar diferentes alternativas hasta encontrar un diseño que encajara mejor con la aplicación.

Al utilizar Claude, se mencionaron referencias de las aplicaciones Plant Parent, Plant Master y Plantì, estudiadas previamente durante el análisis de mercado. En particular, la mayor inspiración provino de **Plantì**, aunque se consideró que su interfaz presentaba una gran cantidad de elementos visuales y de información en pantalla.

Por ello, se optó por adaptar algunas de sus ideas a un diseño más minimalista y coherente con Plant Guardian. Teniendo esto en cuenta, se obtuvieron varias propuestas para mejorar la pantalla de inicio, de las cuales la última fue la seleccionada, realizando posteriormente algunos ajustes en sus elementos para alinearlos con el estilo visual de la aplicación.



4.3. Datos y gestión usuario

Para estas pantallas se aplicó la idea de gamificación y de mostrar el progreso del usuario de una forma visual y gradual, evitando sobrecargar la interfaz con demasiados elementos, tal y como se ha explicado en el apartado anterior.

Una de las pantallas principales es la de “Mi perfil”, la cual permite visualizar información relacionada con el usuario, como el nombre, el icono de perfil, la experiencia acumulada y los logros obtenidos. Parte de esta información puede modificarse desde la pantalla de configuración del usuario, donde es posible editar los datos introducidos durante el registro, a excepción de la contraseña.

Además de guardar los cambios realizados, esta pantalla también incluye la opción de cerrar sesión. Al hacerlo, el usuario es redirigido nuevamente a la pantalla de inicio de sesión, ya que el uso de la aplicación requiere obligatoriamente una cuenta registrada.

Por otro lado, el icono de perfil funciona como una recompensa asociada al progreso del usuario dentro de la aplicación. A medida que el usuario sube de nivel y obtiene más experiencia, se desbloquean nuevas opciones de iconos de perfil disponibles para seleccionar. Para ello, se optó por diseños sencillos inspirados en animales, manteniendo así una estética coherente y amigable con el resto de la aplicación.

De este modo, las pantallas relacionadas con el perfil y la configuración permiten integrar elementos básicos de gamificación sin comprometer la simplicidad y claridad de la experiencia de usuario.

4.4. Añadir planta

Este flujo de pantallas desarrolla una de las funcionalidades principales de la aplicación, relacionada directamente con el primer objetivo del proyecto: la identificación, diagnóstico y personalización del cuidado de una planta.

El proceso fue diseñado de forma simple y guiada, con el objetivo de que el usuario pudiera completarlo de manera intuitiva y sin dificultades. Para ello, se estructuró en

varias pantallas consecutivas que acompañan al usuario durante todo el proceso de registro de una nueva planta.

Esta sección es una de las que más interacción tiene con el backend, ya que realiza llamadas tanto a la API de Trefle como a la API de Gemini. Por un lado, Trefle se utiliza para identificar la planta y obtener información relacionada con sus características y necesidades de cuidado. Por otro lado, Gemini se encarga de generar diagnósticos y recomendaciones personalizadas a partir de la información obtenida y del contexto del usuario.

Además, durante este flujo se desarrollaron diversos componentes visuales que posteriormente se reutilizan en otras partes de la aplicación, especialmente en las pantallas de detalle y seguimiento de las plantas. Esto permitió mantener una coherencia visual entre las distintas secciones de la aplicación y facilitar la reutilización de componentes dentro del desarrollo frontend.

4.5. Mis plantas

En las pantallas de visualización de plantas, la pantalla principal ha sufrido una ligera modificación respecto al diseño original. El diseño inicial proponía un sistema de navegación mediante *tabs* [15], que permitía alternar entre una lista de plantas y una sección de colecciones. Sin embargo, con el objetivo de agilizar la búsqueda de plantas, se optó por sustituir este enfoque por un sistema basado en buscador [16].

Esta decisión se tomó tras analizar los resultados de las entrevistas realizadas a usuarios, en las que se observó que la mayoría disponía de entre 2 y más de 10 plantas (tal y como se ha mencionado en el apartado 3.1. Por ello, aunque algunos usuarios pueden llegar a tener una gran cantidad de plantas, se consideró que en la mayoría de los casos una estructura basada en listas con búsqueda sería más eficiente y directa.

Además, este cambio también permitió simplificar la implementación de funcionalidades y reducir el número de pantallas necesarias dentro de la aplicación. De este modo, se descartó finalmente el sistema de *tabs*, el cual llegó a implementarse de forma inicial, pero fue posteriormente eliminado.

Como resultado, la pantalla final quedó compuesta por un buscador [17] y una lista de plantas, lo que mejora la claridad visual y agiliza el acceso a cada elemento. Además, esto permite centrar la experiencia del usuario en las pantallas de detalle de cada planta, donde se muestra la información relevante de forma más completa.

Estas pantallas de detalle se han diseñado siguiendo el estilo definido en Figma, con ligeras modificaciones en la organización de algunos elementos, pero manteniendo la estructura general prevista inicialmente.

5. Repaso general

Este apartado ofrece una visión general sobre la evolución del proyecto en su etapa final, analizando el cumplimiento de los plazos y la efectividad de la metodología aplicada.

5.1. Resultados obtenidos

Los resultados obtenidos en esta fase muestran la evolución del proyecto desde un prototipo inicial hasta un Producto Mínimo Viable (MVP) ya funcional.

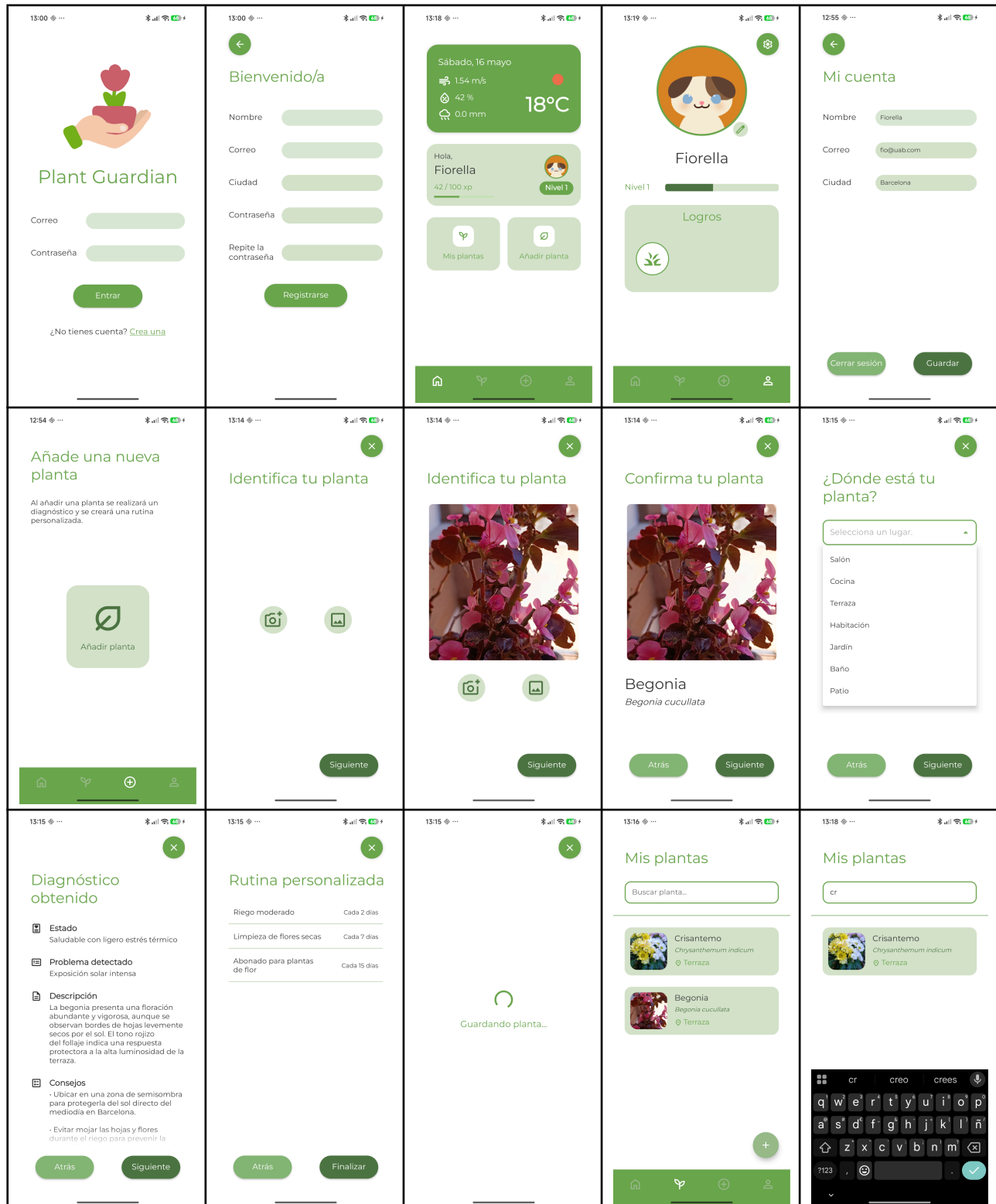
Este avance se ha logrado principalmente durante los últimos *sprints*, donde el objetivo ha sido pasar de pantallas estáticas a una aplicación completamente operativa con lógica real y conexión con el backend. Resumidamente, se han conseguido los siguientes avances principales:

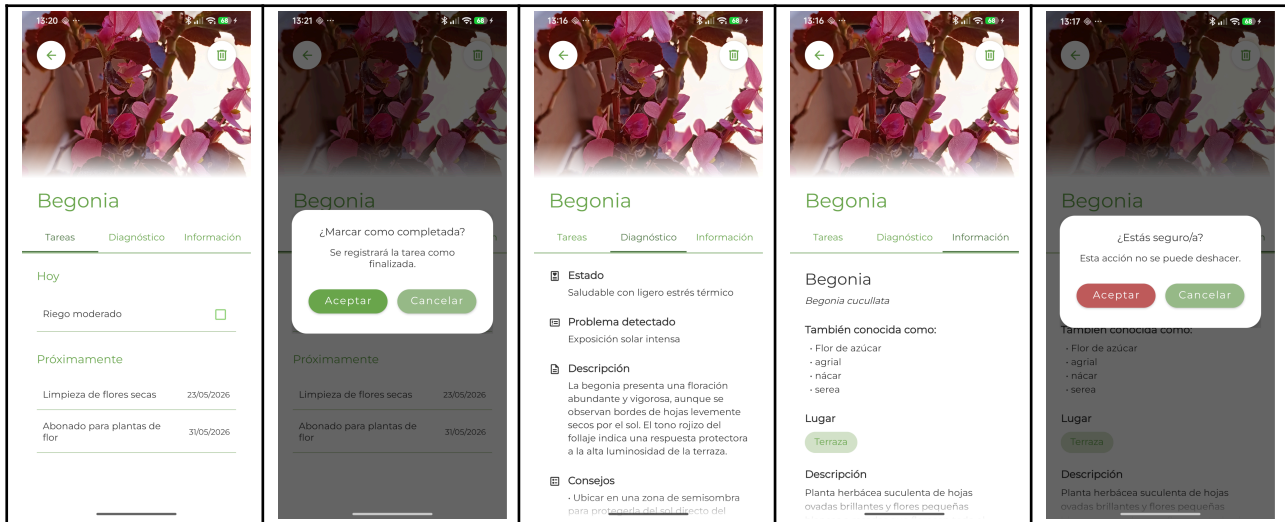
- **Operatividad:** Se ha pasado de un diseño principalmente visual y de un código que se validaba de forma aislada mediante el *framework* FastAPI, a una aplicación conectada con el *backend*. Las pantallas no son sólo interfaces, sino que son elementos funcionales los cuales realizan llamadas a la APIs y guardan los datos directamente en la base de datos de Supabase cuando es necesario.
- **Datos en tiempo real:** Siguiendo la arquitectura explicada en el [apartado 3.3](#), ahora la aplicación maneja los datos de forma dinámica. Esto significa que el usuario no solo ve información, sino que puede interactuar con ella. Por ejemplo, puede gestionar sus plantas y ver cómo se actualiza al momento su progreso (XP y niveles) dentro de la app.
- **Tiempo de respuesta:** Se ha detectado que las APIs pueden tardar más de lo esperado en algunos momentos, especialmente debido al uso de versiones gratuitas con limitaciones de consultas. Por ello, se decidió implementar *progress bars* para informar al usuario de que la aplicación sigue procesando la solicitud y mejorar así la experiencia de uso.
- **Cumplimiento de requisitos:** Los resultados actuales confirman que se han cumplido los requisitos funcionales del [apartado 3.2.1](#), especialmente en aspectos como la actualización de la interfaz según el estado de las tareas (pendientes o completadas), lo que hace que la app sea útil según los objetivos planeados.

En resumen, el proyecto ha evolucionado desde un prototipo inicial hasta una aplicación funcional MVP, con una correcta integración entre *frontend* y *backend*. Ahora, las pantallas son totalmente operativas, conectadas a las APIs y a la base de datos de Supabase, permitiendo la gestión de datos en tiempo real y la interacción directa del usuario con la información. Todo esto completa los objetivos planificados inicialmente, junto con los requisitos funcionales.

5.2. Pantallas implementadas

A continuación, se muestran las pantallas resultantes de la aplicación, junto con los distintos aspectos mencionados anteriormente. En cuanto al diseño final, se ha buscado mantener una interfaz simple y fácil de utilizar para el usuario, utilizando elementos minimalistas y una paleta de colores claros.





5.3. Conclusiones

Tras la finalización de las fases principales de desarrollo y la integración de los sistemas descritos en este informe, se pueden extraer varias conclusiones sobre el estado y la viabilidad del proyecto:

- **Arquitectura:** La arquitectura utilizada (Kotlin en el *frontend* y FastAPI con Supabase en el *backend*) se ha mostrado adecuada, estable y escalable para este MVP, permitiendo una gestión eficiente de los datos en tiempo real.
- **Gamificación:** El sistema de gamificación basado en niveles y XP ha demostrado ser funcional. Sin embargo, todavía queda pendiente validar de forma más completa si cumple su objetivo principal de mejorar la experiencia de usuario y fomentar la interacción y la constancia en el uso de la aplicación.
- **Objetivo de la aplicación:** La aplicación ha evolucionado hacia un sistema de asistencia más completo, pasando de interfaces estáticas a un entorno dinámico en el que el usuario interactúa directamente con sus plantas y tareas. Esto refuerza el objetivo principal del proyecto, que es ofrecer una herramienta útil y activa para el cuidado de plantas.
- **Metodología:** La metodología basada en sprints ha sido clave para alcanzar este punto, ya que ha permitido un desarrollo ordenado y la corrección progresiva de errores. El proyecto se encuentra en un estado funcional, habiendo cumplido los objetivos principales planteados inicialmente.

En resumen, el proyecto Plant Guardian ha alcanzado un estado funcional y avanzado, con una arquitectura sólida y escalable basada en Kotlin, FastAPI y Supabase. Como siguiente paso, se realizará una fase de *testing* y validación de la aplicación, con el objetivo de comprobar los requisitos no funcionales pendientes (rendimiento, disponibilidad y modo *offline*) y evaluar la experiencia de usuario, especialmente en el sistema de gamificación. Esta fase permitirá obtener conclusiones más completas sobre la calidad y viabilidad de la aplicación final.

6. Inteligencia artificial

La inteligencia artificial utilizada durante este sprint ha sido principalmente **Gemini** y **Claude**. Ambas herramientas han servido como apoyo durante distintas fases del desarrollo, tanto a nivel técnico como visual.

Por un lado, Gemini se utilizó principalmente para ayudar en la toma de decisiones tecnológicas y, especialmente, para comprender cómo estructurar correctamente la organización del proyecto en Android Studio. Gracias a esta herramienta, fue posible entender qué clases, fragmentos y pantallas debían implementarse, así como la forma en la que debían relacionarse entre sí dentro de la arquitectura de la aplicación. Una vez comprendida esta estructura, se reutilizó el mismo enfoque en el resto de funcionalidades desarrolladas posteriormente.

Por otro lado, Claude, ya mencionado durante el proceso de rediseño de la pantalla principal, se utilizó como apoyo en la organización de elementos visuales y en el diseño de las interfaces. Principalmente, ayudó a comprender mejor el funcionamiento de animaciones, transiciones y otros efectos visuales aplicados a los distintos componentes de la aplicación. Además, permitió explorar diferentes distribuciones visuales antes de trasladarlas al código definitivo.

El uso de estas herramientas permitió reducir la sobrecarga durante el desarrollo de las pantallas. A medida que avanzaba el proyecto, las funcionalidades se volvieron más complejas y la integración de nuevos *endpoints*, la creación de clases, el diseño de fragmentos y la conexión con el *backend* incrementaban la dificultad del desarrollo. En este contexto, las herramientas de inteligencia artificial facilitaron parte del proceso, agilizando tareas de apoyo técnico y permitiendo centrar el esfuerzo en la implementación funcional del MVP planteado desde el inicio del proyecto.

En conclusión, ambas inteligencias artificiales han servido como herramientas de asistencia durante el desarrollo, ayudando a generar código base, comprender la lógica y estructura del *frontend* e implementar correctamente la integración con el *backend*.

7. Webgrafía

- [1] *Principios de UX para Crear Apps Móviles Intuitivas* - Troop Software Factory. (s. f.). Recuperado 10 de abril de 2026, de <https://www.troopsf.com/principios-ux-apps-intuitivas/>
- [2] *What is User Interface (UI) Design? — updated 2026* | IxDF. (s. f.). Recuperado 10 de abril de 2026, de <https://ixdf.org/literature/topics/ui-design>
- [3] *What is Gamification? — updated 2026* | IxDF. (s. f.). Recuperado 10 de mayo de 2026, de <https://ixdf.org/literature/topics/gamification>
- [4] *CRUD (Crear, Leer, Actualizar, Eliminar)* | Microsoft Learn. (s. f.). Recuperado 22 de abril de 2026, de <https://learn.microsoft.com/es-es/iis-administration/api/crud>
- [5] *Docs + Quickstarts* | Render. (s. f.). Recuperado 15 de mayo de 2026, de <https://render.com/docs>
- [6] *Free cronjobs - from minutely to once a year.* - cron-job.org. (s. f.). Recuperado 15 de mayo de 2026, de <https://cron-job.org/en/>
- [7] *Wake up Render free instances* | FastCron. (s. f.). Recuperado 15 de mayo de 2026, de <https://www.fastcron.com/tutorials/render-cron/>
- [8] *venv — Creation of virtual environments — Python 3.14.5 documentation.* (s. f.). Recuperado 9 de marzo de 2026, de <https://docs.python.org/3/library/venv.html>
- [9] *Cómo consumir una API desde una aplicación Android* - Tutoriales. (s. f.). Recuperado 11 de abril de 2026, de <https://coderslink.com/talento/blog/como-consumir-una-api-desde-una-aplicacion-android>
- [10] *Introduction* | Retrofit. (s. f.). Recuperado 11 de abril de 2026, de <https://square.github.io/retrofit/>
- [11] *Adapter.* (s. f.). Recuperado 8 de mayo de 2026, de <https://refactoring.guru/es/design-patterns/adapter>
- [12] *Descripción general de ViewModel | App architecture | Android Developers.* (s. f.). Recuperado 8 de mayo de 2026, de <https://developer.android.com/topic/libraries/architecture/viewmodel?hl=es-419>
- [13] *Observer.* (s. f.). Recuperado 7 de mayo de 2026, de <https://refactoring.guru/es/design-patterns/observer>
- [14] *Coil.* (s. f.). Recuperado 11 de abril de 2026, de <https://coil-kt.github.io/coil/>
- [15] *Tabs – Material Design 3.* (s. f.). Recuperado 9 de mayo de 2026, de <https://m3.material.io/components/tabs/overview>

[16] *Search Box vs. Navigation (Video)* - NN/G. (s. f.). Recuperado 9 de mayo de 2026, de <https://www.nngroup.com/videos/search-box-vs-navigation/>

[17] *Cómo crear una interfaz de búsqueda | Views | Android Developers*. (s. f.). Recuperado 10 de mayo de 2026, de <https://developer.android.com/develop/ui/views/search/search-dialog?hl=es-419>